

BREAKING APP

Mobile Application (Android APK) — AI Development Prompt Document

DOCUMENT PURPOSE

This document is a comprehensive, structured prompt for an AI coding assistant to build the Android mobile application for 'Breaking App' — a news bulletin platform. Follow every section precisely. Do not skip any feature. Ask for clarification only if something is ambiguous.

1. PROJECT OVERVIEW

Build an Android mobile application (APK) called 'Breaking App'. It is a news bulletin platform where users can browse, search, filter, save, and share news updates. The app should be built using React Native so it can be compiled into an Android APK.

 Use React Native as the framework. The output must be a working Android APK.

2. TECH STACK

- Framework: React Native (target Android APK)
- Language: JavaScript / TypeScript
- Navigation: React Navigation
- State Management: Redux Toolkit or Zustand
- HTTP Client: Axios
- Push Notifications: Firebase Cloud Messaging (FCM)
- Local Storage: AsyncStorage (for saved news, user preferences)
- Media: React Native Video + Image Slider component
- Sharing: React Native Share API
- Backend Communication: REST API (JSON responses)

3. SCREENS & FEATURES

3.1 Splash Screen & Onboarding

- Show app logo on launch
- Brief onboarding (1-3 slides) explaining the app — shown only on first launch
- Auto-navigate to Home Screen after splash

3.2 Home Screen — News Feed

The main screen. Displays a vertical scrollable list of news cards fetched from the backend API.

Each News Card must show:

- Photo or video (with slider support if multiple images/videos are attached)
- News title
- Short description / content preview
- Category name (styled as a colored badge)
- Published date (formatted: e.g., '29 May 2026')
- Hashtags (tappable, e.g., #BreakingNews)
- Reporter/Admin profile image (circular avatar)
- Reporter/Admin name
- Share button (icon, triggers share sheet)
- Bookmark/Save icon (toggleable)

Advertisement Cards:

- Ads appear inline between news cards
- Default placement: after every 4th news card
- Ad card shows a banner image
- Tapping the ad opens a URL in the device browser

 *Ad positions are configurable from the admin panel. Fetch ad config along with news feed from API.*

3.3 Share Feature

When the user taps the share button on a news card, trigger the native share sheet with:

- First image of the news
- News title
- Short description
- App download/open link (deep link or Play Store link)

Supported platforms (handled by native share sheet):

- WhatsApp
- Facebook
- Instagram
- Twitter / X
- Telegram
- Native device sharing (any app the user has)

3.4 Category Filter

A horizontal scrollable filter bar at the top of the news feed. Categories are fetched dynamically from the API.

- 'All' tab selected by default

- Tapping a category filters the news feed to show only that category's news
- Categories include (but are not limited to): Politics, Sports, Technology, Entertainment, Business
- Each category can have a custom background color and font color (set by admin)

3.5 Hashtag Filter

Users can filter news by hashtag. Accessible from a dedicated 'Hashtags' filter section or by tapping a hashtag on a news card.

- Hashtags are fetched from the API
- Examples: #BreakingNews, #Trending, #Elections, #Cricket
- Tapping a hashtag shows all news tagged with that hashtag

3.6 City Filter

Users can filter news by city.

- Multi-city selection is supported (user can select more than one city)
- News feed updates to show news from selected cities
- City list is fetched from the API
- 'All Cities' is the default selection

3.7 Search Screen

A dedicated search screen accessible from a search icon in the top navigation bar.

- Search input field (auto-focus on open)
- Search works across: News title, Keywords in description, Category name, Hashtags
- Results display in the same card format as the news feed
- Show 'No results found' message when nothing matches

3.8 Saved / Bookmarked News

A dedicated screen accessible from the bottom navigation or menu.

- Displays all news the user has saved/bookmarked
- Each item has a 'Remove' option to unsave
- Saved data is stored locally (AsyncStorage) AND synced to the user account if logged in
- Persists across app sessions

3.9 News Detail Screen

Opens when a user taps a news card.

- Full image/video gallery (swipeable slider)
- Full news title
- Full description / content (supports rich text)
- Category badge, hashtags, city
- Published date and time

- Reporter profile image and name
- Share button
- Bookmark/Save button
- Back navigation

3.10 Push Notifications

Integrate Firebase Cloud Messaging (FCM) to receive push notifications.

- Users receive notifications for: Breaking news, Trending news, Category-based updates
- Tapping a notification navigates directly to the relevant news detail screen (deep link)
- Notification permission requested on first launch

3.11 Bottom Navigation Bar

Persistent bottom navigation with tabs:

- Home (News Feed)
- Search
- Saved
- Notifications / Profile

4. API INTEGRATION

All data is fetched from a REST API backend. The base URL will be provided via an environment variable (e.g., `REACT_APP_API_URL`). Implement the following API calls:

Endpoint	Method	Purpose
/api/news	GET	Fetch paginated news feed (supports filters: category, hashtag, city, search query)
/api/news/:id	GET	Fetch single news item details
/api/categories	GET	Fetch all active categories
/api/hashtags	GET	Fetch all hashtags
/api/cities	GET	Fetch all available cities
/api/ads	GET	Fetch active advertisements and their positions
/api/user/saved	GET / POST / DELETE	Get, save, or remove bookmarked news
/api/notifications/register	POST	Register FCM device token

💡 Implement pagination / infinite scroll for the news feed (e.g., page & limit query params).

5. STATE MANAGEMENT

- News list, filters (category, hashtag, city, search) — global state
- Saved/bookmarked news — persisted in AsyncStorage + synced with API
- Advertisement config — fetched on app load, stored in state
- User session / auth token — stored securely (Keychain/AsyncStorage)

6. UI / UX REQUIREMENTS

- Clean, modern news app design (reference apps: BBC News, Inshorts)
- Category badge colors come from the API (admin-configured)
- Image slider with dot indicators for multiple media
- Skeleton loaders while content is loading
- Empty state illustrations for: no saved news, no search results
- Pull-to-refresh on news feed
- Error handling with user-friendly messages ('Could not load news. Tap to retry.')
- Smooth screen transitions

7. PERFORMANCE REQUIREMENTS

- Images loaded lazily (only when visible)
- News list uses FlatList with proper keyExtractor
- API responses cached where appropriate
- App should work smoothly on mid-range Android devices (Android 8+)

8. FILE / FOLDER STRUCTURE (Recommended)

Organize the project as follows:

- /src/screens — All screen components
- /src/components — Reusable UI components (NewsCard, AdCard, CategoryFilter, etc.)
- /src/navigation — Navigation stack/tab configuration
- /src/store — Redux/Zustand store and slices
- /src/api — Axios instance and API call functions
- /src/utils — Helpers (date formatting, share logic, etc.)
- /src/constants — Colors, fonts, API base URL
- /src/assets — Images, icons, fonts

9. DELIVERABLES

- Full React Native project source code
- Working Android APK build
- README with setup, build, and environment variable instructions
- All environment variables documented in a .env.example file

💡 *Do NOT hard-code the API base URL. Use environment variables only.*